# Machine Learning Review

Last modified: 2026-10-25

```
# The Python chunks on this page import these; reticulate provisions them.
# Attaching reticulate gives the inline R its `py` object.
library(reticulate)
py_require(c("numpy", "pandas", "matplotlib"))
```

The order of topics in these notes follows Brian Hutchinson's CSCI 481/581, used with his permission; his machine learning[1] review module is condensed here into a page that links to lds. The exercises are new.

## 1 Welcome

### 1.1 Plan for this lecture

Deep learning reuses the pieces of supervised learning from DATA 371 and DATA 471, which lds[2] covers in full. This lecture reviews three of them on the handwritten digits from the course overview, then shows where each one reappears in deep learning:

1. the parts of a learning problem: a set of candidate rules, the parameters[3] that pick one out, and a loss function[4];
2. why models are trained on a smooth loss rather than on their error rate[5];
3. why a model is judged on data it was not fit to.

### 1.2 The digits again

The exercises here use the 1,797 handwritten digits that the course overview loads from the UCI Machine Learning Repository[6] [1], from the same local copy, `data/optdigits-test.csv`[7]. They also use two of its features[8], each an average of an 8 by 8 image's pixels, with rows and columns counted from 0:

- center ink, over rows 2 to 5 and columns 3 and 4;
- left ink, over rows 2 to 5 and columns 0 to 2.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

digits = pd.read_csv("data/optdigits-test.csv", header=None)
pixels = digits.iloc[:, :64].to_numpy()
labels = digits[64].to_numpy()
images = pixels.reshape(-1, 8, 8)
```

[1] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-machine-learning
[2] https://morrison-lab.github.io/lds/
[3] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[4] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-loss-function
[5] https://morrison-lab.github.io/lds/chapters/metrics.html#def-error-rate
[6] https://archive.ics.uci.edu/dataset/80
[7] https://morrison-lab.github.io/dds/chapters/data/optdigits-test.csv
[8] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature

```
center_ink = images[:, 2:6, 3:5].mean(axis=(1, 2))
left_ink = images[:, 2:6, 0:3].mean(axis=(1, 2))
print(images.shape, center_ink[:5], left_ink[:5])
```

```
(1797, 8, 8) [ 0.375 15.875 11.375  8.25   9.125] [5.16666667 2.25       3.58333333 0.41666667 4.
```

# 2 The parts of a learning problem

## 2.1 Candidate rules and their parameter

**Exercise 2.1** (What the 0-or-1 rule chose among)**.** The course overview's rule for 0 or 1 calls an image a 1 when its center ink is above a cutoff, and found its cutoff by trying every value from 0 to 16 in steps of 0.5. What set of rules did that search choose among?

Solution

*Solution* 2.1. Every rule of the form "a 1 when the center ink is above $\theta$, and a 0 otherwise", one rule for each number $\theta$. The number $\theta$ that picks one rule out of this set is the rule's one parameter[a]. A deep network picks one function out of a much larger set, with many parameters[b] instead of one.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

## 2.2 The loss

**Exercise 2.2** (How the 0-or-1 rule chose)**.** By what criterion did the search of Exercise 2.1 pick one cutoff over another?

Solution

*Solution* 2.2. It kept the cutoff that got the fewest images wrong. Counting mistakes uses the 0-1 loss function[a]: 1 for a wrong label and 0 for a right one. Its average over the images is the rule's error rate[b], which is the rule's empirical risk[c] under that loss. Choosing the parameter[d] with the lowest empirical risk is empirical risk minimization[e], and deep networks are fit the same way.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-loss-function
[b] https://morrison-lab.github.io/lds/chapters/metrics.html#def-error-rate
[c] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-empirical-risk
[d] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[e] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-empirical-risk-minimization

# 3 Losses to train on

## 3.1 The error rate

**Exercise 3.1** (The error rate against the cutoff)**.** Among the 0s and 1s, plot the error rate[a] of the 0-or-1 rule against its cutoff $\theta$, for every $\theta$ from 0 to 16 in steps of 0.01.

[a] https://morrison-lab.github.io/lds/chapters/metrics.html#def-error-rate

Solution

*Solution* 3.1.

```
is_01 = (labels == 0) | (labels == 1)
ink_01 = center_ink[is_01]
is_one = labels[is_01] == 1

cutoffs = np.arange(0, 16.005, 0.01)
error_rate = np.array([np.mean((ink_01 > c) != is_one) for c in cutoffs])

fig, ax = plt.subplots(figsize=(6, 3), layout="constrained")
ax.plot(cutoffs, error_rate)
ax.set_xlabel(r"cutoff $\theta$ on center ink")
ax.set_ylabel("error rate")
plt.show()
```

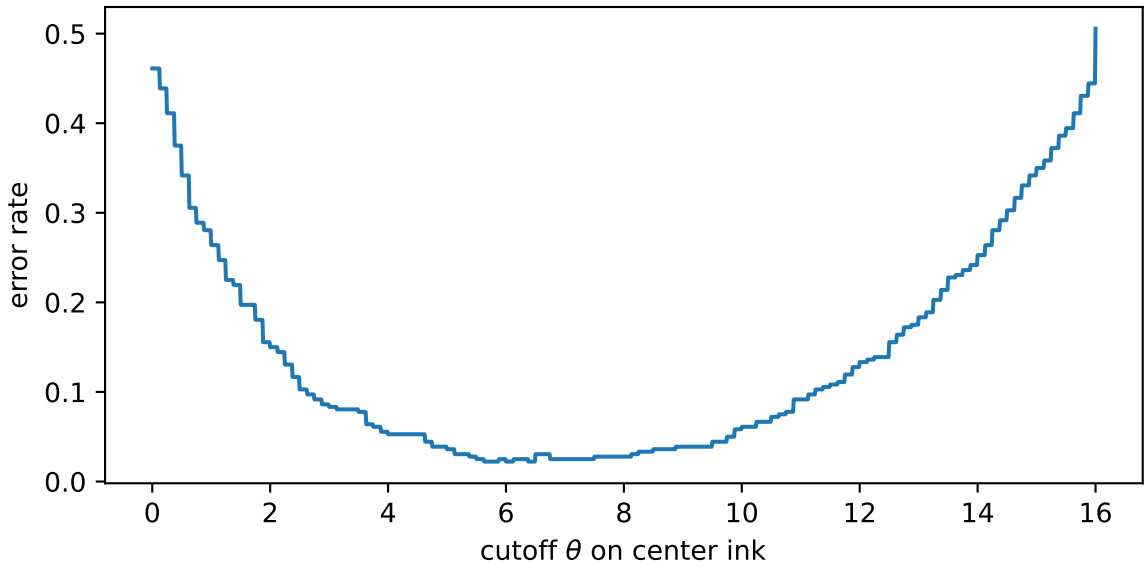



Figure 1: The error rate[a] of the 0-or-1 rule among the 360 images of 0s and 1s, against the cutoff.

[a] https://morrison-lab.github.io/lds/chapters/metrics.html#def-error-rate

**Exercise 3.2** (Fitting by following the error rate)**.** Could gradient descent[a] find the best cutoff by following the slope of the curve in Figure 1?

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#the-algorithm

Solution

*Solution* 3.2.

```
lowest = error_rate == error_rate.min()
best_cutoffs = cutoffs[lowest]
# A new stretch of lowest error starts wherever `lowest` turns from False to True.
n_stretches = np.sum(np.diff(lowest.astype(int)) == 1) + int(lowest[0])
print(error_rate.min(), best_cutoffs.min(), best_cutoffs.max(), n_stretches)
```

```
0.022222222222222223 5.63 6.49 3
```

```
print(len(np.unique(ink_01)))
```

```
99
```

No. The curve in Figure 1 is a staircase. The error rate[a] changes only where $\theta$ crosses one of the 99 values that the center ink takes, and it is flat everywhere in between, so its slope is 0 wherever the slope exists. Gradient descent would never move.

The grid of cutoffs in steps of 0.01 reaches the lowest error rate, 2.2% (8 of the 360 images), on 3 separate stretches between 5.63 and 6.49, and it found them by trying each cutoff in turn. That works for one parameter[b], but a network has thousands of weights, and trying every combination of their values is out of reach.

[a] https://morrison-lab.github.io/lds/chapters/metrics.html#def-error-rate
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

## 3.2 A smooth loss

**Exercise 3.3** (The log loss against the cutoff)**.** Swap the 0-or-1 rule for a soft version, which gives each image a probability of being a 1:

$$\hat{\pi} = \text{expit}(x - \theta),$$

where

- $x$ is the image's center ink,
- $\theta$ is the cutoff, the rule's one parameter[a],
- expit is the logistic function[b], $\text{expit}(z) = 1/(1 + \exp\{-z\})$,
- and $\hat{\pi}$ is the probability the rule gives that the image is a 1.

Among the 0s and 1s, plot the mean log loss[c] of this rule against $\theta$, over the cutoffs of Exercise 3.1.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[b] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#exm-logistic-function
[c] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#def-log-loss

Solution

*Solution* 3.3.

```python
def mean_log_loss(c):
    """Mean log loss of the soft 0-or-1 rule with cutoff `c`."""
    p = 1 / (1 + np.exp(-(ink_01 - c)))
    return -np.mean(np.where(is_one, np.log(p), np.log(1 - p)))

log_loss = np.array([mean_log_loss(c) for c in cutoffs])

fig, ax = plt.subplots(figsize=(6, 3), layout="constrained")
ax.plot(cutoffs, log_loss)
ax.set_xlabel(r"cutoff $\theta$ on center ink")
ax.set_ylabel("mean log loss")
plt.show()
```

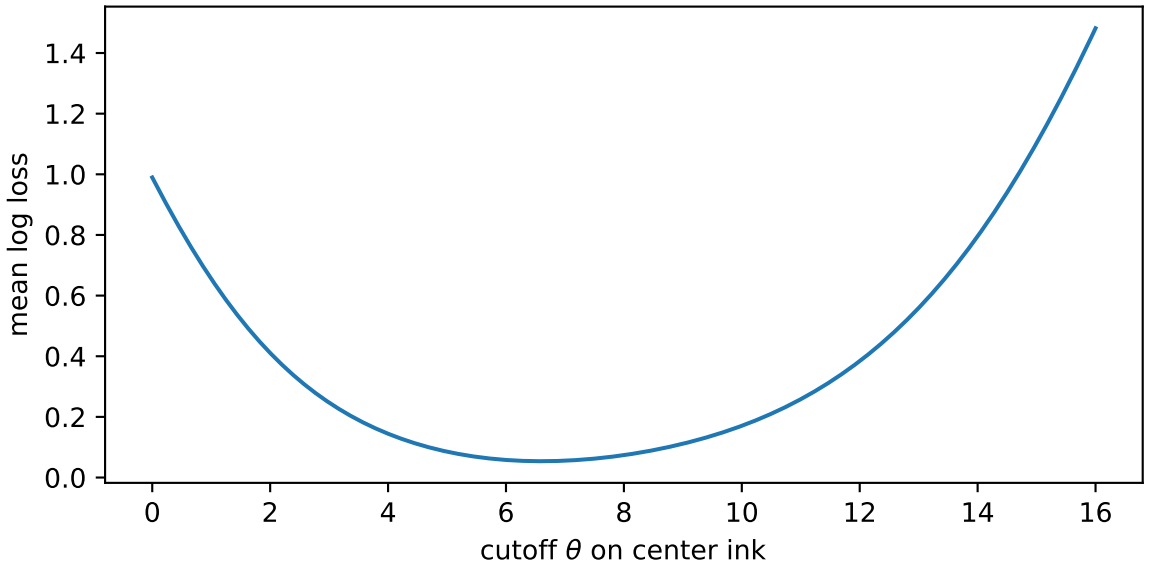



Figure 2: The mean log loss[a] of the soft 0-or-1 rule among the images of 0s and 1s, against the cutoff.

[a] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#def-log-loss

**Exercise 3.4** (Fitting by following the log loss)**.** Could gradient descent[a] find the best cutoff by following the slope of the curve in Figure 2?

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#the-algorithm

Solution

*Solution* 3.4.

```python
c_log = cutoffs[log_loss.argmin()]
error_at_c_log = np.mean((ink_01 > c_log) != is_one)
print(c_log, log_loss.min(), error_at_c_log)
```

```
6.6000000000000005 0.053815807670509946 0.030555555555555555
```

Yes. The curve in Figure 2 is smooth, with one lowest point. Its slope is nonzero at every other cutoff, so a step against the slope always goes downhill, and gradient descent has a direction to follow.

The lowest log loss[a] is at $\theta = 6.60$. The hard rule with that cutoff gets 11 of the 360 images of 0s and 1s wrong, against 8 for the best cutoffs in Solution 3.2. Minimizing the log loss does not minimize the error rate[b], but, unlike the error rate, it can be minimized by gradient descent.

Deep networks are trained the same way: on a smooth loss such as the log loss, which is also called

cross-entropy[c], while their error rate or accuracy[d] is what gets reported.

[a] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#def-log-loss
[b] https://morrison-lab.github.io/lds/chapters/metrics.html#def-error-rate
[c] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#def-log-loss
[d] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy

# 4 Judging a model

## 4.1 Held-out data

**Exercise 4.1** (The digit-average rule on new images)**.** The course overview's digit-average rule labels an image with the digit whose average (center ink, left ink) is nearest, and it was scored on the same images the averages came from. Split the 1,797 images at random into two halves, a training set[a] and a test set[b]. Compute the digit averages from the training set only. What share of each set does the rule get right?

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-test-set

Solution

*Solution* 4.1.

```
rng = np.random.default_rng(481)
order = rng.permutation(len(labels))
train, test = order[: len(labels) // 2], order[len(labels) // 2 :]
hand_features = np.column_stack([center_ink, left_ink])

def accuracy(predicted, actual):
    return np.mean(predicted == actual)

def nearest(points, references):
    """Index of the nearest row of `references` to each row of `points`."""
    distances = ((points[:, None, :] - references[None, :, :]) ** 2).sum(axis=2)
    return distances.argmin(axis=1)

means = np.array(
    [hand_features[train][labels[train] == k].mean(axis=0) for k in range(10)]
)
acc_mean_train = accuracy(nearest(hand_features[train], means), labels[train])
acc_mean_test = accuracy(nearest(hand_features[test], means), labels[test])
print(len(train), len(test), round(acc_mean_train, 3), round(acc_mean_test, 3))
```

```
898 899 0.468 0.485
```

The rule gets 46.8% of the training set[a] right and 48.5% of the test set[b]. The two shares are close: ten digit averages, twenty numbers in all, leave the rule little room to fit the training set's quirks, so its training error[c] says roughly what its test error[d] will be. The test share came out a little higher only by the luck of this split; another random split could put it a little lower.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-test-set
[c] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-error
[d] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-test-error

**Exercise 4.2** (A rule that memorizes)**.** Now make the rule far more flexible: label each image with the digit of the training image nearest to it in (center ink, left ink). This is one-nearest-neighbor[a] classification. Using the split from Solution 4.1, what share of the training set[b] and of the test set[c] does it get right?

[a] https://morrison-lab.github.io/lds/chapters/non-parametric-models.html#k-nearest-neighbors
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set
[c] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-test-set

Solution

*Solution* 4.2.

```
train_features = hand_features[train]
acc_nn_train = accuracy(
    labels[train][nearest(train_features, train_features)], labels[train]
)
acc_nn_test = accuracy(
    labels[train][nearest(hand_features[test], train_features)], labels[test]
)
print(round(acc_nn_train, 3), round(acc_nn_test, 3))
```

```
0.964 0.415
```

The rule gets 96.4% of the training set[a] right, but only 41.5% of the test set[b], worse than the 48.5% of the digit-average rule in Solution 4.1. On the training set, each image's nearest neighbor is usually itself, so the rule mostly recalls labels it has already seen. It falls short of 100% only where images of different digits share both feature[c] values.

This rule has learned the training images themselves rather than what the digits look like, which lds calls overfitting[d]. Deep networks are flexible enough to memorize their training sets too, even when the labels are assigned at random [2], so these notes judge every network on images it was not trained on.

[a] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-training-set
[b] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-test-set
[c] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature
[d] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-overfitting

# 5 What carries over

## 5.1 From one cutoff to a deep network

Table 1: Each piece of a learning problem on this page, and its counterpart in a deep network.

| Piece | On this page | In a deep network |
|---|---|---|
| Candidate rules | Cutoff rules on one feature[9] | Networks of one architecture |
| Parameters[10] | The cutoff $\theta$ | The weights of every layer |
| Training loss | Log loss[11] of the soft rule | Cross-entropy[12], the log loss of a softmax[13] output |
| Fitting | A search over a grid of cutoffs | Stochastic gradient descent[14] on minibatches of the loss |

[9] https://morrison-lab.github.io/lds/chapters/course-overview.html#def-feature
[10] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter
[11] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#def-log-loss
[12] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#def-log-loss
[13] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#sec-multiclass
[14] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#sec-sgd

| Piece | On this page | In a deep network |
|---|---|---|
| Judging | Accuracy[15] of the digit-average and nearest-neighbor rules on a held-out test set[16] | Accuracy on held-out images |

The math these pieces rest on, derivatives and gradients, matrices, and probability, is reviewed in lds's math prerequisites[17] and probability[18] chapters.

> **Read**
>
> Chapter 5 of *Deep Learning*, Machine Learning Basics[a], reviews these pieces at more length [3, Ch. 5].
>
> [a] https://www.deeplearningbook.org/contents/ml.html

## References


[1] E. Alpaydin and C. Kaynak, "Optical recognition of handwritten digits." UCI Machine Learning Repository, 1998. doi: 10.24432/C50P49[19].

[2] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning requires rethinking generalization," in *International conference on learning representations*, 2017. Available: https://arxiv.org/abs/1611.03530

[3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016. Available: https://www.deeplearningbook.org/


[15] https://morrison-lab.github.io/lds/chapters/metrics.html#def-accuracy
[16] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-test-set
[17] https://morrison-lab.github.io/lds/chapters/math-prereqs.html
[18] https://morrison-lab.github.io/lds/chapters/probability.html
[19] https://doi.org/10.24432/C50P49